# UNIT-V
# PROBABLISITIC MODELS

## Probablisitic Models:

- Probabilistic models, allows us to handle inherent uncertainty in data and make informed predictions.
- Probabilities can be useful to express a model's expectation about the class of a given instance.
- These models make predictions based on probability distributions, rather than absolute values, allowing for a more nuanced and accurate understanding of complex systems.
- Probabilistic models are used in a variety of machine learning tasks such as classification, regression, clustering, and dimensionality reduction.
- Some popular probabilistic models include:
  - ✓ Naïve Bayes
  - ✓ Bayesian Networks
  - ✓ Gaussian Mixture Models (GMMs)
  - ✓ Hidden Markov Models (HMMs)
  - ✓ Markov Random Fields (MRFs)
- Probabilistic models are used in various applications such as image and speech recognition, natural language processing, and recommendation systems.
- In recent years, significant progress has been made in developing probabilistic models that can handle large datasets efficiently.
- Probabilistic models are mainly classified into two categories. They are:
  - ✓ Generative models
  - ✓ Discriminative.

**Generative models:**

- Generative models aim to model the ***joint distribution*** *P (Y, X)* of the input and output variables. (*Note:- X* is the feature vector and *Y* is a target variable)
- Once we know this joint distribution we can derive any conditional or marginal distribution involving the same variables..
- These models are powerful because they can generate new data that resembles the training data based on the probability distribution of the original dataset.
- They can be used for tasks such as image and speech synthesis, language translation, and text generation.
- ***Conditional Probability:*** The probability of the occurrence of an event X given that an event Y has already occurred is called the conditional probability of X given Y and is denoted by P(X|Y).

  We have, $P(X|Y) = \frac{P(X \cap Y)}{P(Y)}$ $if\ P(Y) \neq 0$
- Bayes Theorem to calculate the posterior probability as follows:

$$posterior = \frac{prior \times likelihood}{evidence}$$

$$P(Y|X) = \frac{P(Y,X)}{P(X)}$$

$$P(Y|X) = \frac{P(Y).P(X|Y)}{P(X)}$$

Where,

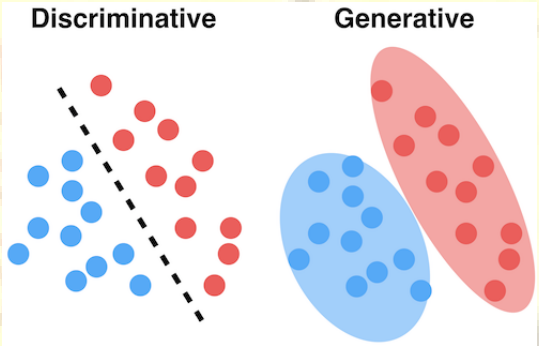
- P(Y) is called the prior probability of proposition.

- P(X) is called the prior probability of evidence.
- P(Y|X) is called the posterior probability of Y given X.
- P(X|Y) is called the likelihood of X given Y.

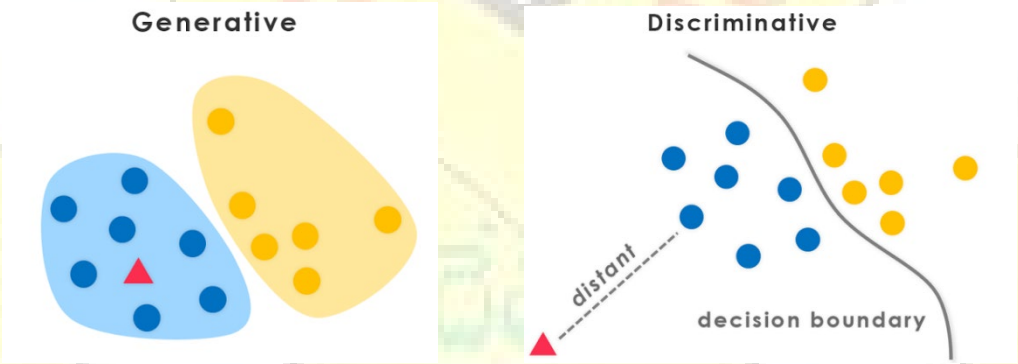
- Alternatively, generative models can be described by the likelihood function P(X|Y), since P(Y,X) = P(X|Y)P(Y) and the target or prior distribution can be easily estimated.
- If there is a presence of outliers in the dataset, then it affects these types of models to a significant extent.

**Discriminative models:**

- Discriminative models aim to model the ***conditional distribution*** i.e., **posterior probability distribution** P(Y|X) of the output variable given the input variable. (*Note:- X* is the feature vector and *Y* is a target variable)
- Discriminative models focus on modeling the decision boundary between classes in a classification problem.
- These models focus on accurate predictions rather than generating new data.




- They can be used for tasks such as image recognition, speech recognition, and sentiment analysis.
- If we have some outliers present in the dataset, discriminative models work better compared to generative models.
- However, one major drawback of these models is the ***misclassification problem***, i.e., wrongly classifying a data point.




## Normal Distribution and its Geometric Interpretations:

**Normal Distribution (Gaussian Distribution):**

- Normal Distribution is the most common or normal form of distribution of Random Variables.
- It is also known as **Gaussian** or **Gauss** distribution in Statistics or Probability.
- It is a probability distribution that is symmetric about the mean, showing that data near the mean are more frequent in occurrence than data far from the mean.
- It is characterized by a ***bell-shaped curve*** that is symmetrical around the mean point.

- ***Mathematical Definition:-*** A continuous random variable "x" is said to follow a normal distribution with parameter μ(mean) and σ(standard deviation), if it's ***probability density function*** is given by,

$$y = f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Where,

x is a random variable.

$\mu$ is the mean or location parameter.

$\sigma$ is the standard deviation or scale parameter.

- Some examples of normal distribution include Distribution of Height of People, Distribution of Errors in any Measurement, Distribution of Blood Pressure of any Patient, etc.

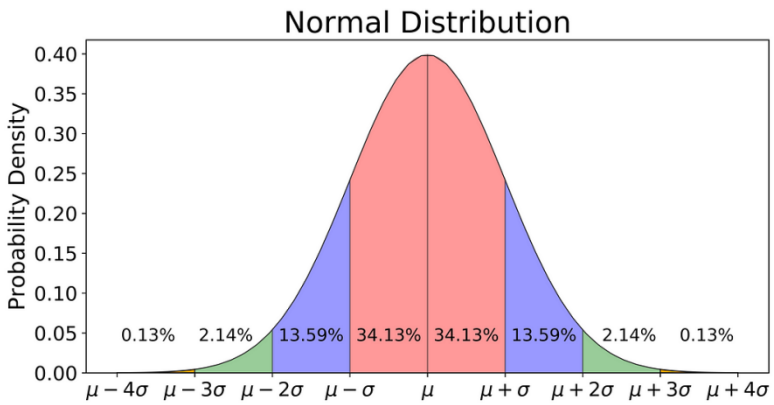



***Fig.:-*** Empirical Formula of Normal Distribution

- The empirical rule is a handy quick estimate of the data's spread given the mean and standard deviation of a data set that follows a normal distribution. It states that:
  - ✓ 68.26% of the data will fall within 1 standard deviation of the mean(μ±1σ)
  - ✓ 95.44% of the data will fall within 2 standard deviations of the mean(μ±2σ)
  - ✓ 99.7% of the data will fall within 3 standard deviations of the mean(μ±3σ)

***Properties of Normal Distribution:***

- ✓ For normal distribution of data, mean, median, and mode are equal, (i.e., Mean = Median = Mode).
- ✓ The total area under the normal distribution curve is equal to 1.
- ✓ It is symmetrical about the mean. Each half of the distribution is a mirror image of the other half.
- ✓ It asymptotically approaches the horizontal axis.
- ✓ It is a Unimodal Curve, i.e. a curve with only one peak.

***Applications:***

- ✓ Models like **Linear Discriminant Analysis (LDA)**, **Gaussian Naive Bayes**, **Logistic Regression**, and **Linear Regression** assume normality.
- ✓ **Sigmoid functions** work naturally with normally distributed data.
- ✓ Many natural phenomena follow a log-normal distribution, making it relevant for financial data and forecasting.

**Normal Distribution and its Geometric Interpretations:**

- We can draw a connection between probabilistic and geometric models by considering probability distributions defined over Euclidean spaces.
- The most common such distributions are normal distributions, also called Gaussians.

- It recalls the most important facts concerning univariate and multivariate normal distributions.
- Let's start by considering the univariate, two-class case. Suppose the values of x ∈ R follow a mixture model: i.e., each class has its own probability distribution (a component of the mixture model).
- We will assume a Gaussian mixture model, which means that the components of the mixture are both Gaussians. We thus have

$$P(x|\oplus) = \frac{1}{\sqrt{2\pi}\sigma^{\oplus}}\ exp^{-\frac{1}{2}\left[\frac{(x-\mu^{\oplus})}{\sigma^{\oplus}}\right]^2}$$

$$P(x|\ominus) = \frac{1}{\sqrt{2\pi}\sigma^{\ominus}}\ exp^{-\frac{1}{2}\left[\frac{(x-\mu^{\ominus})}{\sigma^{\ominus}}\right]^2}$$

where $\mu^{\oplus}$ and $\sigma^{\oplus}$ are the mean and standard deviation for the positive class, and $\mu^{\ominus}$ and $\sigma^{\ominus}$ are the mean and standard deviation for the negative class. This gives the following likelihood ratio:

$$LR(x) = \frac{P(x|\oplus)}{P(x|\ominus)} = \frac{\sigma^{\oplus}}{\sigma^{\ominus}}\ exp^{\left(-\frac{1}{2}\left[\left(\frac{(x-\mu^{\oplus})}{\sigma^{\oplus}}\right)^2-\left(\frac{(x-\mu^{\ominus})}{\sigma^{\ominus}}\right)^2\right]\right)}$$

## Naïve Bayes Classifier:

- ✓ Naïve Bayes algorithm is a supervised learning algorithm, which is based on **Bayes theorem** and used for solving classification problems.
- ✓ Naïve Bayes Classifier is one of the simple and most effective classification algorithm which helps in building the fast machine learning models that can make quick predictions.
- ✓ It belongs to the family of generative learning algorithms.
- ✓ So, Naïve Bayes is a probabilistic classifier that predicts the probability of an input belonging to a particular class.
- ✓ It assumes ***feature independence***, which simplifies the model and makes it computationally efficient.
- ✓ Some popular examples of Naïve Bayes Algorithm are ***spam filtration, medical diagnosis, recommendation systems, credit scoring, weather prediction*** and ***classifying articles***.
- ✓ It is mainly used in ***text classification*** that includes a high-dimensional training dataset.

**Bayes' Theorem:**

- ✓ Bayes' theorem is used to determine the probability of a hypothesis with given prior knowledge. It depends on the conditional probability.
- ✓ The formula for Bayes' theorem is given as:

$$P(h \mid D) = \frac{P(D \mid h) \cdot P(h)}{P(D)}$$

Where,

*P(h|D)* is the ***posterior probability***: Probability of hypothesis *h* given the data *D*.

*P(D|h)* is the ***likelihood probability***: Probability of data *D* given hypothesis h is true.

*P(h)* is the ***prior probability***: Probability of the hypothesis *h* before observing the evidence.

*P(D)* is the ***marginal probability***: Prior probability of the data *D*.

Bayes rule for multiple independent variable X is defined as:

$$P(Y=k|X) = \frac{P(X|Y=k) * P(Y=k)}{P(X)}, Where\ X = (x_1, x_2, \dots, x_n)\ and\ k\ is\ the\ Class\ of\ Y$$

**Naive Bayes algorithms for learning and classifying text:**

LEARN_NAIVE_BAYES_TEXT (*Examples*, *V*)

*Examples is a set of text documents along with their target values. V is the set of all possible target values. This function learns the probability terms $P(w_k|v_j,)$, describing the probability that a randomly drawn word from a document in class $v_j$ will be the English word $w_k$. It also learns the class prior probabilities $P(v_j)$.*

1. *collect all words, punctuation, and other tokens that occur in Examples*
   - *Vocabulary* ← the set of all distinct words and other tokens occurring in any text document from *Examples*
2. *calculate the required $P(v_j)$ and $P(w_k|v_j)$ probability terms*
   - For each target value $v_j$ in *V* do
     1. $docs_j$ ← the subset of documents from *Examples* for which the target value is $v_j$
     2. $P(v_j) \leftarrow \frac{|docs_j|}{Examples}$
     3. $Text_j$ ← a single document created by concatenating all members of $docs_j$
     4. $n$ ← total number of distinct word positions in $Text_j$
     5. for each word $w_k$ in *Vocabulary*
        - $n_k$ ← number of times word $w_k$ occurs in $Text_j$
        - $P(w_k|v_j) \leftarrow \frac{n_k+1}{n+|Vocabulary|}$

CLASSIFY_NAIVE_BAYES_TEXT (*Doc*)

*Return the estimated target value for the document Doc. $a_i$ denotes the word found in the $i^{th}$ position within Doc.*

- *positions* ← all word positions in *Doc* that contain tokens found in *Vocabulary*
- Return $v_{NB}$, *where*

$$v_{NB} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j) \prod_{i \in positions} P(a_i|v_j)$$

**Working of Naïve Bayes' Classifier:**

Suppose we have a dataset of **weather conditions** and corresponding target variable "**Play**".

So using this dataset we need to decide that whether we should play or not on a particular day according to the weather conditions. So to solve this problem, we need to follow the below steps:

**Step 1:** Convert the given dataset into frequency tables.

| Sl. No. | Outlook | Play |
|---|---|---|
| 0 | Rainy | Yes |
| 1 | Sunny | Yes |
| 2 | Overcast | Yes |
| 3 | Overcast | Yes |
| 4 | Sunny | No |
| 5 | Rainy | Yes |
| 6 | Sunny | Yes |
| 7 | Overcast | Yes |
| 8 | Rainy | No |
| 9 | Sunny | No |
| 10 | Sunny | Yes |
| 11 | Rainy | No |
| 12 | Overcast | Yes |
| 13 | Overcast | Yes |

**Step 2:** Generate Likelihood table by finding the probabilities (conditional and prior) of given features. Creating Likelihood table by finding the probabilities like:

Overcast probability = 0.29 and probability of playing is 0.64.

| Weather | Yes | No |
|---|---|---|
| Overcast | 5 | 0 |
| Rainy | 2 | 2 |
| Sunny | 3 | 2 |
| **Total** | **10** | **4** |

| Weather | No | Yes | |
|---|---|---|---|
| Overcast | 0 | 5 | 5/14 = 0.36 |
| Rainy | 2 | 2 | 4/14 = 0.29 |
| Sunny | 2 | 3 | 5/14 = 0.36 |
| All | 4/14 = 0.29 | 10/14 = 0.71 | |

**Step 3:** Now, use Naive Bayesian equation to calculate the posterior probability.

Now, use Naive Bayesian equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of the prediction.

***Applying Bayes' theorem:***

$$P(Yes|Sunny) = \frac{P(Sunny|Yes) * P(Yes)}{P(Sunny)}$$

Here, we have P (Sunny |Yes) = 3/10 = 0.3, P(Sunny) = 0.36, P(Yes)= 10/14 = 0.71

$$P(Yes|Sunny) = \frac{0.3 * 0.71}{0.36} = 0.59$$

$$P(No|Sunny) = \frac{P(Sunny|No) * P(No)}{P(Sunny)}$$

Here, we have P (Sunny |No) = 2/4 = 0.5, P(Sunny) = 0.36, P(No)= 4/14 = 0.29

$$P(No|Sunny) = \frac{0.5 * 0.29}{0.36} = 0.40$$

**Step 4:** Select the class with the highest posterior probability as the predicted class for the new sample. So, from above calculation we see that $P(Yes|Sunny) > P(No|Sunny)$

Hence, on a Sunny day, Player can play the game.

**Types of Naïve Bayes Model:**

There are three types of Naive Bayes Model, which are given below:

- ✓ **Gaussian**: The Gaussian model assumes that features follow a normal distribution. This means if predictors take continuous values instead of discrete, then the model assumes that these values are sampled from the Gaussian distribution.
- ✓ **Multinomial**: The Multinomial Naïve Bayes classifier is used when the data is multinomial distributed. It is primarily used for document classification problems, it means a particular document belongs to which category such as Sports, Politics, education, etc. The classifier uses the frequency of words for the predictors.
- ✓ **Bernoulli**: The Bernoulli classifier works similar to the Multinomial classifier, but the predictor variables are the independent Booleans variables. Such as if a particular word is present or not in a document. This model is also famous for document classification tasks.

**Advantages of Naïve Bayes Classifier:**

- ✓ Naïve Bayes is easy to implement and efficient to predict a class of datasets.
- ✓ It can be used for both binary and multi-class Classifications.
- ✓ It can handle high dimensional data.
- ✓ It performs well in the presence of categorical features.
- ✓ For numerical features data is assumed to come from normal distributions

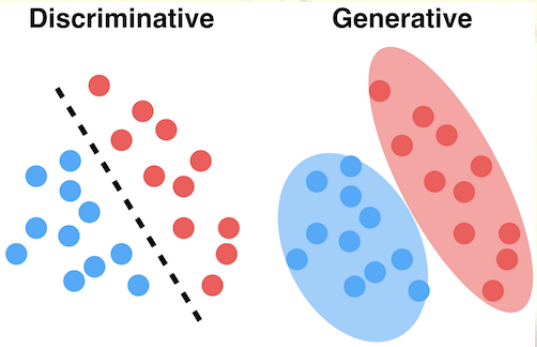
**Disadvantages of Naïve Bayes Classifier:**

- ✓ Assumes that features are independent, so it cannot learn the relationship between features.
- ✓ Can be influenced by irrelevant attributes.
- ✓ May assign zero probability to unseen events, leading to poor generalization. To avoid this issue, laplace smoothing can be leveraged.

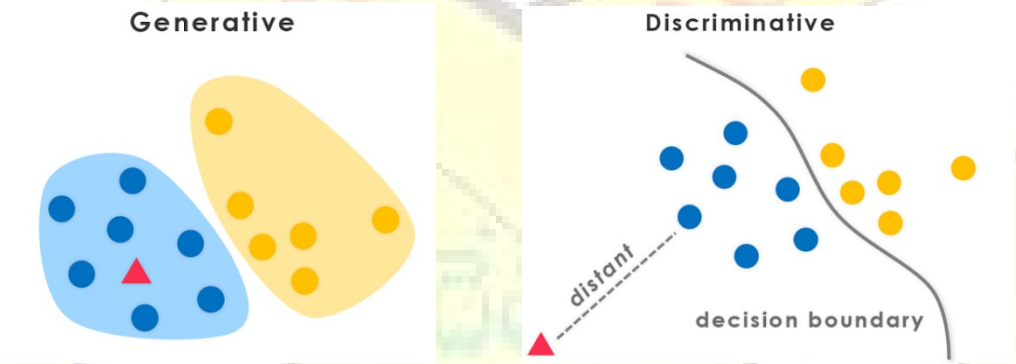
## Discriminative learning with Maximum likelihood:

**Discriminative models:**

- Discriminative models aim to model the ***conditional distribution*** i.e., **posterior probability distribution** P(Y|X) of the output variable given the input variable. (*Note:- X* is the feature vector and *Y* is a target variable)
- Discriminative models focus on modeling the decision boundary between classes in a classification problem.
- These models focus on accurate predictions rather than generating new data.




- They can be used for tasks such as image recognition, speech recognition, and sentiment analysis.
- If we have some outliers present in the dataset, discriminative models work better compared to generative models.
- However, one major drawback of these models is the ***misclassification problem***, i.e., wrongly classifying a data point.




**Discriminative learning by optimising conditional likelihood:**

- One of the most commonly used discriminative models is logistic regression.
- The easiest way to understand logistic regression is as a linear classifier whose probability estimates have been logistically calibrated using the method of obtaining probabilities from linear classifiers but with one crucial difference: calibration is an integral part of the training algorithm, rather than a post-processing step.
- While in generative models the decision boundary is a by-product of modelling the distributions of each class, logistic regression models the decision boundary directly.

- For example, if the classes are overlapping then logistic regression will tend to locate the decision boundary in an area where classes are maximally overlapping, regardless of the 'shapes' of the samples of each class. This results in decision boundaries that are noticeably different from those learned by generative classifiers.

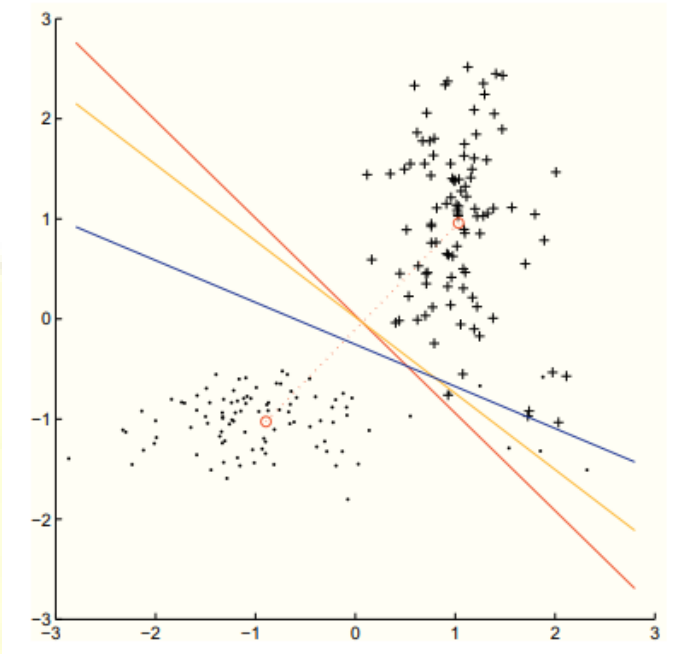



***Fig.:-*** On this data set, logistic regression (in blue) outperforms the basic linear classifier (in red) and the least squares classifier (in orange) because the latter two are more sensitive to the shape of the classes, while logistic regression concentrates on where the classes overlap.

- The likelihood ratio is expressed as $\exp\big(\gamma(d(x) - d_0)\big)$ with $d(x) = w.x - t$.
- Since we are learning the parameters all at once in discriminative learning, we can absorb $\gamma$ and $d_0$ into $w$ and $t$.
- So the logistic regression model is simply given by

$$\hat{p}(x) = \frac{\exp(w.x - t)}{\exp(w.x - t) + 1} = \frac{1}{1 + \exp(-(w.x - t))}$$

- Assuming the class labels are $y = 1$ for positives and $y = 0$ for negatives, this defines a Bernoulli distribution for each training example:

$$P(y_i|x_i) = \hat{p}(x_i)^{y_i}(1 - \hat{p}(x_i))^{(1-y_i)}$$

- It is important to note that the parameters of these Bernoulli distributions are linked through w and $t$, and consequently there is one parameter for every feature dimension, rather than for every training instance.
- The likelihood function is

$$CL(w,t) = \prod_i P(y_i|x_i) = \prod_i \hat{P}(x_i)^{y_i}\left(1 - \hat{P}(x_i)\right)^{(1-y_i)}$$

- This is called conditional likelihood to stress that it gives us the conditional probability $P(y_i\,|x_i)$ rather than $P(x_i)$ as in a generative model.
- Notice that our use of the product requires the assumption that the $y$-values are independent given $x$; but this is an entirely reasonable assumption and not nearly as strong as the naive Bayes assumption of $x$ being independent within each class.

## Probabilistic Models with Hidden variables:

- Suppose you are dealing with a four-class classification problem with classes *A, B, C* and *D*.
- If you have a sufficiently large and representative training sample of size *n*, you can use the relative frequencies in the sample $n_A, \ldots, n_D$ to estimate the class prior $\hat{P}_A = \frac{n_A}{n}, \ldots, \hat{p}_D = \frac{n_D}{n}$, as we have done many times before.
- Conversely, if you know the prior and want to know the most likely class distribution in a random sample of n instances, you would use the prior to calculate expected values $\mathbb{E}[n_A] = p_A.n, \ldots, \mathbb{E}[n_D] = p_D.n$.
- So, complete knowledge of one allows us to estimate or infer the other.
- However, sometimes we have a bit of knowledge about both. For example, we may know that $p_A = 1/2$ and that *C* is twice as likely as *B*, without knowing the complete prior. And we may know that the sample we saw last week was evenly split between $A \cup B$ and $C \cup D$, and that *C* and *D* were equally large, but we can't remember the size of *A* and *B* separately. What should we do?
- Formalising what we know about the prior, we have $p_A = 1/2$; $p_B = \beta$, as yet unknown; $p_C = 2\beta$, since it is twice $p_B$ ; and $p_D = 1/2 - 3\beta$, since the four cases need to add up to 1.
- Furthermore: $n_A + n_B = a + b = s$, $n_C = c$ and $n_D = d$, with *s*, *c* and *d* known. We want to infer *a*, *b* and $\beta$: however, it seems we are stuck in a chicken-and egg problem. If we knew $\beta$ we would have full knowledge about the prior and we could use that to infer expected values for *a* and *b*:

$$\frac{\mathbb{E}[a]}{\mathbb{E}[b]} = \frac{1/2}{\beta} \qquad \mathbb{E}[a] + \mathbb{E}[b] = s$$

From which we could derive:

$$\mathbb{E}[a] = \frac{1}{1+2\beta}s \qquad \mathbb{E}[b] = \frac{2\beta}{1+2\beta}s$$

So, for example, if $s = 20$ and $\beta = 1/10$, then $\mathbb{E}[a] = 16\frac{2}{3}$ and $\mathbb{E}[b] = 3\frac{1}{3}$

- Conversely, if we knew *a* and *b*, then we could estimate $\beta$ by maximum-likelihood estimation, using a multinomial distribution for *a*, *b*, *c* and *d*:

$$P(a,b,c,d|\beta) = K(1/2)^a \beta^b (2\beta)^c (1/2 - 3\beta)^d$$

$$\ln P(a,b,c,d|\beta) = \ln K + a\ln(1/2) + b\ln\beta + c\ln(2\beta) + d\ln(1/2 - 3\beta)$$

- Here, *K* is a combinatorial constant that doesn't affect the value of $\beta$ which maximises the likelihood. Taking the partial derivative with respect to $\beta$ gives

$$\frac{\partial}{\partial\beta}\ln P(a,b,c,d|\beta) = \frac{b}{\beta} + \frac{2c}{2\beta} - \frac{3d}{1/2 - 3\beta}$$

- Setting to 0 and solving for $\beta$ finally gives

$$\hat{\beta} = \frac{b+c}{6(b+c+d)}$$

- So, for example, if $b = 5$ and $c = d = 10$, then $\hat{\beta} = \frac{1}{10}$.
- The way out of this chicken-and-egg problem is to iterate the following two steps:
  i. calculate an expected value of the missing frequencies *a* and *b* from an assumed or previously estimated value of the parameter $\beta$; and
  ii. calculate a maximum-likelihood estimate of the parameter β from assumed or expected values of the missing frequencies *a* and *b*.
- These two steps are iterated until a stationary configuration is reached.

## Estimation-Maximization Methods:

- The EM (Estimation -Maximization) algorithm was proposed and named in a seminal paper published in 1977 by Arthur Dempster, Nan Laird, and Donald Rubin.
- The EM algorithm combines various unsupervised machine learning techniques (i.e., K-means) used to find or obtain **maximum likelihood estimates (MLE)** or **maximum a posteriori estimates (MAP)** for observable and unobservable variables in statistical models.
- Therefore, the EM algorithm is especially useful when dealing with **latent variables** and **missing data** that are not directly observable but can be inferred from other observed variables.
- It is also referred to as the ***latent variable model***.
- A latent variable model consists of both observable and unobservable variables where observable can be predicted while unobserved are inferred from the observed variable.
- The EM algorithm is also the basis for many unsupervised clustering algorithms.

**Expectation-Maximization (EM) Algorithm:**

- *A general technique for finding maximum likelihood estimators in latent variable models is the estimation-maximization (EM) algorithm.*
- The EM algorithm operates through an iterative process, consisting of two steps until convergence:
  1. ***Estimation (E) - step***: In the E-step (*guess*), the algorithm computes the latent variables i.e. expectation of the log-likelihood using the current parameter estimates.
  2. ***Maximization (M) - step***: In the M-Step, the algorithm determines the parameters that maximize the expected log-likelihood obtained in the E-step and the corresponding model parameters are then updated based on the estimated latent variables.

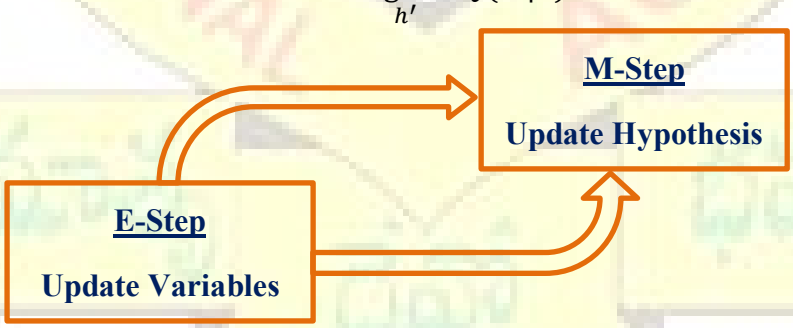
(Or)

1. ***Estimation (E) - step***: Calculate $Q(h'|h)$ using the current hypothesis $h$ and the observed data $X$ to estimate the probability distribution over $Y$.

$$Q(h'|h) \leftarrow E[\ln P(Y|h')|h, X]$$

2. ***Maximization (M) - step***: Replace hypothesis $h$ by the hypothesis $h'$ that maximizes this $Q$ function.

$$h \leftarrow \underset{h'}{\operatorname{argmax}}\, Q(h'|h)$$




**What is Convergence in the EM algorithm?**

- ✓ ***Convergence is defined as the specific situation in probability based on intuition***, e.g., if there are two random variables that have very less difference in their probability, then they are known as converged.

**How Expectation-Maximization (EM) Algorithm Works:**

- ✓ The EM algorithm is completed mainly in 4 steps, which include ***Initialization Step, Expectation Step, Maximization Step, and convergence Step***. These steps are as follows:

  **Step-1:** ***Initialization:*** Initially, a set of initial values of the parameters are considered. A set of incomplete observed data is given to the system with the assumption that the

observed data comes from a specific model.

***Step-2: E-Step:*** In this step, we use the observed data in order to estimate or guess the values of the missing or incomplete data. It is basically used to update the variables.

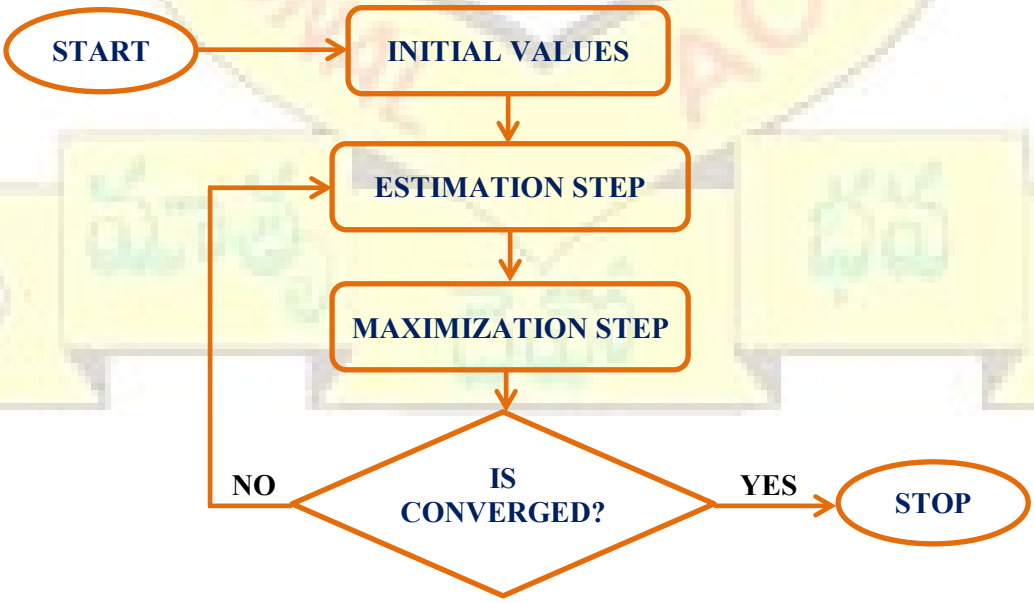
- Compute the posterior probability or responsibility of each latent variable given the observed data and current parameter estimates.
- Estimate the missing or incomplete data values using the current parameter estimates.
- Compute the log-likelihood of the observed data based on the current parameter estimates and estimated missing data.

***Step-3: M-step:*** In this step, we use the complete data generated in the preceding "Estimation" – step in order to update the values of the parameters. It is basically used to update the hypothesis.

- Update the parameters of the model by maximizing the expected complete data log-likelihood obtained from the E-step.
- This typically involves solving optimization problems to find the parameter values that maximize the log-likelihood.
- The specific optimization technique used depends on the nature of the problem and the model being used.

***Step-4: Convergence***: In this step, it is checked whether the values are converging or not, if yes, then stop otherwise repeat *step-2* and *step-3* i.e., E–step and M–step until the convergence occurs.

- Check for convergence by comparing the change in log-likelihood or the parameter values between iterations.
- If the change is below a predefined threshold, stop and consider the algorithm converged.
- Otherwise, go back to the E-step and repeat the process until convergence is achieved.




**Applications of EM Algorithm**:

- ✓ ***Clustering:*** EM is frequently used in clustering scenarios where data point assignments to clusters are uncertain or ambiguous.

- ✓ ***Handling Missing Data:*** When data is incomplete or certain features are unobservable, EM helps predict missing values.
- ✓ ***Probabilistic Models***: EM is valuable for estimating parameters in probabilistic models with hidden variables

**Advantages:**

- ✓ Effective for handling missing data and latent variables.
- ✓ Robustness in various scenarios.
- ✓ Convergence guarantees like K-means for a wide class of probabilistic models.

**Disadvantages:**

- ✓ It is sensitive to initialization and converges slowly.
- ✓ It may converge to local optimum that is dependent on the initial configuration.

## Gaussian Mixtures/ Gaussian Mixture Models:

- **Gaussian Mixture Models (GMMs)** are a powerful technique used in machine learning for clustering and density estimation.
- A GMM is a probabilistic model that represents a dataset as a mixture of several Gaussian distributions (also known as normal distributions).
- It assumes that the data points come from different Gaussian distributions, and each distribution represents a cluster.
- Unlike K-means clustering, which assigns each point to a single cluster; Gaussian Mixture Models are probabilistic models and use the soft clustering approach for distributing the points in different clusters.
- It is widely used in Clustering applications (identifying subpopulations within a dataset), Density estimation (modeling complex data distributions), Anomaly detection (detecting outliers) and pattern recognition applications.

**Gaussian Distribution (Normal Distribution):**

- In real-life datasets, many variables follow a Gaussian distribution (univariate or multivariate).
- The probability density function (PDF) of a univariate (or one dimensional space) Gaussian distribution is given by:

$$p(x|\mu,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Where,

x- is a data point

$\mu$- is the mean of distribution

$\sigma^2$- is the variance

- For multivariate Gaussian distributions (d-variate), the probability density function (PDF) is more complex, involving a mean vector and a covariance matrix.
- For Multivariate (let us say d-variate) Gaussian Distribution, the probability density function is given by

$$p(X|\mu,\Sigma) = \frac{1}{2\pi^{d/2}\sqrt{|\Sigma|}} \exp\left(-\frac{1}{2}(X-\mu)^T\Sigma^{-1}(X-\mu)\right)$$

Where $X$ is the input vector, $\mu$ is a d-dimensional vector denoting the mean of the distribution and $\Sigma$ is the $d \times d$ covariance matrix.

- Hence, for a dataset with $d$ features, we would have a mixture of $k$ Gaussian distributions (where $k$ is equivalent to the number of clusters), each having a certain mean vector and variance matrix.

- But the mean and variance value for each Gaussian are determined using a technique called Expectation-Maximization (EM).

**Gaussian Mixture Model**

- Suppose there are (K) clusters (known in advance).
- So, each cluster has its own mean ($\mu$) and covariance matrix ($\sigma$).
- The mixing coefficient ($\pi$) represents the weight of each cluster.
- The probability density function of Gaussian Mixture Model is a linear combination of the densities of all (K) Gaussian distributions:

$$G(X) = \sum_{k=1}^{K} \pi_k \cdot N(x \mid \mu_k, \Sigma_k)$$

**Expectation-Maximization (EM) Algorithm:**

- GMM parameters cannot be estimated directly due to the presence of hidden variables (cluster assignments).
- To estimate the parameter of the Gaussian Mixture Model (GMM), the most popular technique is **Maximum Likelihood Estimation**.
- The EM algorithm iteratively estimates the parameters:
  - ✓ ***Expectation Step (E-step):*** Compute the posterior probabilities of each data point belonging to each cluster.
  - ✓ ***Maximization Step (M-step):*** Update the parameters using the expected cluster assignments.
- Repeat E-step and M-step until convergence.

## Compression based Models:

- Compression-based model is both closely related to and quite distinct from the probabilistic approach.
- Consider the maximum a posteriori decision rule:

$$y_{MAP} = \underset{y}{\operatorname{argmax}}\, P(X = x|Y = y)P(Y = y)$$

| *Y* | *P*(lottery=1\|Y) | *IC*(lottery=(1\|Y) | *P*(lottery=1\|Y) | *IC*(lottery=(1\|Y) |
|---|---|---|---|---|
| **spam** | 0.40 | **1.32 bits** | 0.60 | **0.74 bits** |
| **ham** | 0.12 | **3.06 bits** | 0.88 | **0.18 bits** |

**Table:-** Example marginal likelihoods

- Taking negative logarithms, we can turn this into an equivalent minimization:

$$y_{MAP} = \underset{y}{\operatorname{argmin}} - \log P(X = x|Y = y) - \log P(Y = y)$$

- This follows because for any two probabilities $0 < p < p' < 1$ we have $\infty > -\log p > -\log p' > 0$.
- If an event has probability $p$ of happening, the negative logarithm of $p$ quantifies the information content of the message that the event has indeed happened.
- This makes intuitive sense, as the less expected an event is, the more information an announcement of the event contains. The unit of information depends on the base of the logarithm: it is customary to take logarithms to the base 2, in which case information is measured in bits.

- For example, if you toss a fair coin once and tell me it came up heads, this contains $-\log_2 1/2 = 1$ bit of information; if you roll a fair die once and let me know it came up six, the information content of your message is $-\log_2 1/6 = 2.6$ bits.
- Equation above tells us that the MAP decision rule chooses the least surprising or the most expected class for an instance x given particular prior distributions and likelihoods. We write $IC(X|Y) = -\log_2 P(X|Y)$ and $IC(Y) = -\log_2 P(Y)$.
- Clearly, for a uniform distribution over $k$ outcomes, each outcome has the same information content $-\log_2 1/k = \log_2 k$. For a non-uniform distribution these information contents differ, and hence it makes sense to compute the average information content or entropy $\sum_{i=1}^{k} -p_i \log_2 p_i$. We have encountered entropy before as an impurity measure.
- So far we have really seen that there is a ***one-to-one relationship*** between ***probability*** and ***information content***.
- Compression-based learning is a fundamental result from information theory proved by Claude Shannon in 1948.
- Shannon's result says – loosely speaking – that we cannot transmit information at a rate that surpasses entropy, but we can get arbitrarily close to the optimal rate by designing clever binary codes.
- Some well-known codes include the Shannon–Fanon code and the Huffman code, which are worth looking up as they employ a simple tree structure to build the code from empirical probabilities.
- Even more efficient codes, such as arithmetic coding, combine multiple messages into a single code word.
- Assuming the availability of a near-optimal code, we can now turn the tables and use information content – or 'description length' as it is more commonly called – as a proxy for probability.
- One simplified version of the minimum description length (MDL) principle runs as follows:
  ***Minimum description length principle:*** Let *L(m)* denote the length in bits of a description of model *m*, and let *L(D|m)* denote the length in bits of a description of data *D* given model *m*. According to the minimum description length principle, the preferred model is the one minimising the description length of model and data given model:
  $$m_{MDL} = \underset{m \in M}{\operatorname{argmin}}(L(m) + L(D|m))$$
- In a predictive learning context, 'description of data given model' refers to whatever information we need, in addition to the model and the feature values of the data, to infer the target labels.
- If the model is 100% accurate no further information is needed, so this term essentially quantifies the extent to which the model is incorrect.
- For example, in a uniform two-class setting we need one bit for every data point incorrectly classified by the model. The term *L(m)* quantifies the complexity of the model.
- For instance, if we are fitting a polynomial to the data we need to encode the degree of the polynomial as well as its roots, up to a certain resolution. MDL learning thus trades off accuracy and complexity of a model: the complexity term serves to avoid overfitting in a similar way to the regularisation term in ridge regression and the slack variable term in soft-margin SVMs.
- What encoding to use in order to determine the model complexity *L(m)* is often not straightforward and to some extent subjective. This is similar to the Bayesian perspective, where we need to define a prior distribution on models. The MDL viewpoint offers a concrete way of defining model priors by means of codes.